Introduction to Intelligent Robotics - Camera Calibration and Stereo Vision Homework

May 2026

Homework. Camera Calibration and Stereo Vision

In this homework, you will implement and analyze a camera calibration and stereo vision pipeline. Please complete two implementation problems and one experimental analysis section:

1. Calibrate a single camera using a planar calibration target.
2. Calibrate a stereo camera pair and estimate depth from rectified images.
3. Analyze calibration accuracy, error sources, and the effect of calibration quality on robotic perception.

Camera calibration is a basic but essential tool for robot perception. The goal is to connect the pinhole camera model taught in class with a practical calibration pipeline that estimates camera intrinsics, distortion parameters, camera poses, stereo extrinsics, disparity, and depth.

You may use OpenCV or other standard computer vision libraries for low-level image processing, corner detection, and nonlinear optimization. However, your report must clearly explain what each estimated parameter means and how the final results are evaluated.

Provided Code and Project Organization

The course will provide a starter codebase for this assignment. The exact file names may vary slightly, but the intended organization is:

- `robotics_perception/camera_model.py`
- `robotics_perception/calibration.py`
- `robotics_perception/stereo_calibration.py`
- `robotics_perception/visualization.py`
- `scripts/single_camera_entry.py`
- `scripts/stereo_entry.py`
- `configs/checkerboard.yaml`
- `sample_data/synthetic/`

You may organize the rest of your project in any way that makes sense to you. There is no required package layout beyond the starter code. Your code should, however, be easy to run, easy to inspect, and reproducible.

At minimum, your project should include code for:

- loading calibration images,
- detecting checkerboard, AprilTag, or ChArUco points,
- constructing 3D calibration target points in metric units,
- estimating camera intrinsic parameters and distortion coefficients,
- estimating per-image camera extrinsics for single-camera calibration,
- estimating stereo extrinsics between the left and right cameras,
- computing reprojection error,
- undistorting images,
- rectifying stereo images,
- computing disparity and depth maps,
- plotting and saving calibration results.

You are encouraged to keep your implementation modular. For example, many students will find it useful to separate image loading, target detection, calibration utilities, visualization, and experiment scripts. This is only a suggestion, not a required structure.

External References

You may consult external documentation and code bases as references, but your submitted implementation and report should clearly reflect your own understanding.

OpenCV documentation. You may use OpenCV calibration, stereo calibration, rectification, and stereo matching functions as implementation references.

Camera calibration references. Zhang's planar calibration method is an optional reference for understanding how homographies can be used to estimate camera parameters.

Marker-based calibration references. If you choose to use AprilTag, ArUco, or ChArUco boards, you may consult the relevant library documentation. You must still explain the target geometry and the meaning of the detected 2D-3D correspondences.

Learning Goals

After finishing this homework, you should be able to:

- Explain the pinhole camera model and camera intrinsic matrix.
- Explain radial and tangential distortion.
- Detect calibration target points and build 2D-3D correspondences.
- Calibrate a single camera and interpret `fx`, `fy`, `cx`, and `cy`.
- Compute and interpret reprojection error.
- Estimate per-image camera poses relative to a calibration target.
- Calibrate a stereo camera pair and interpret the relative rotation and translation.
- Explain essential matrix, fundamental matrix, epipolar lines, and stereo rectification.
- Convert disparity to depth using focal length and baseline.
- Discuss how calibration quality affects robotic perception tasks.

General Coding Requirements

Your submission should make it possible for the grader to reproduce your main results. Include clear instructions for installing dependencies, running single-camera calibration, running stereo calibration, generating visualizations, and reproducing your plots and reported numbers.

Your code should save model parameters, calibration outputs, logs, plots, and visualization images in a readable format. Use random seeds when applicable. In a research or robotics setting, calibration results are only useful if another person can reproduce them and understand how they were produced.

Use fixed and well-documented units. For example, if your checkerboard square size is `0.025` meters, then all reported translations and baselines should also be in meters. If you use millimeters, state this clearly and use the same unit consistently.

Problem 1. Single-Camera Calibration

Goal

Implement and analyze a single-camera calibration pipeline using a planar calibration target, such as a checkerboard, AprilTag board, or ChArUco board.

Given a set of calibration images, your program should detect 2D image points on the calibration target, construct the corresponding 3D points in the target coordinate frame, and estimate the camera intrinsic parameters, distortion coefficients, and per-image camera extrinsics.

Background

The pinhole camera model maps a 3D point in the camera frame to an image pixel using the camera intrinsic matrix:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

For a calibration target point X_B expressed in the board frame, its projection onto the image can be written as:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X_B \\ 1 \end{bmatrix},$$

where K is the intrinsic matrix, R and t are the board-to-camera extrinsic parameters for one image, and (u, v) is the projected pixel coordinate.

In real cameras, lens distortion must also be considered. You should estimate and report radial and tangential distortion parameters.

Tasks

1. Load a set of single-camera calibration images.
2. Detect calibration target corners or keypoints in each image.
3. Construct the corresponding 3D target points in metric units.
4. Estimate the camera intrinsic matrix K.
5. Estimate lens distortion parameters.
6. Estimate camera extrinsics for each calibration image.
7. Undistort example images using the calibrated camera model.
8. Compute the average reprojection error.
9. Save visualizations of detected corners, reprojected points, and undistorted images.

You may use standard computer vision libraries for low-level corner detection and nonlinear optimization. However, your report must explain the camera model, the meaning of each calibrated parameter, and how reprojection error is computed.

Coding Suggestions

Your single-camera calibration code will likely need:

- a configuration file that stores board size and square size,
- an image loader that reads calibration images in a stable order,
- a detector for checkerboard, AprilTag, or ChArUco points,
- a function that creates 3D object points in the calibration board frame,
- a calibration function that returns K, distortion coefficients, rotation vectors, and translation vectors,
- a reprojection-error function,
- visualization code for detected points, reprojected points, and undistorted images.

Start with a small number of images to verify that your point detector works. Then use a larger and more diverse set of viewpoints for the final calibration. Good calibration images should include different target positions, distances, and orientations.

Deliverables

- Source code for single-camera calibration.
- The final intrinsic matrix K.
- The final distortion coefficients.
- Per-image extrinsics, either as rotation vectors and translation vectors or as 4×4 transformation matrices.
- The average reprojection error.
- Example images showing detected calibration points.
- Example images before and after undistortion.
- Short written answers:
  - What do f_x, f_y, c_x, and c_y mean?
  - What distortion parameters did you estimate?
  - What is reprojection error?
  - Is your calibration result good? Why or why not?
  - Which images or viewpoints produced larger errors?

Target

There is no fixed reprojection-error threshold required for full credit because the exact number depends on the camera, target, image resolution, and data quality. However, your result should be reasonable, and your report should explain whether the error is small or large relative to your image resolution and corner-detection quality.

Problem 2. Stereo Camera Calibration and Depth Estimation

Goal

Extend your calibration pipeline to a stereo camera setup. Given synchronized left and right camera images of the same calibration target, estimate the intrinsics of both cameras, the relative pose between the two cameras, and use stereo rectification to compute disparity and depth.

Background

Stereo vision estimates depth by comparing corresponding pixels in two images. After rectification, corresponding points should lie on the same image row. The horizontal pixel shift between the left and right views is the disparity d.

For a rectified stereo pair, depth can be estimated by:

$$Z = \frac{f_x B}{d},$$

where Z is depth, f_x is the focal length in pixels, B is the stereo baseline, and d is disparity. This relationship is one of the most important connections between the camera model and 3D perception.

Tasks

1. Load synchronized left and right calibration images.
2. Detect corresponding calibration target points in both views.
3. Independently calibrate the left and right cameras.
4. Estimate the relative rotation R and translation T between the two cameras.
5. Estimate or discuss the essential matrix E and fundamental matrix F.
6. Perform stereo rectification.
7. Visualize rectified image pairs with horizontal epipolar lines.
8. Compute a disparity map from rectified stereo images.
9. Convert disparity to depth using the stereo camera model.
10. Save disparity and depth visualizations.

Stereo Extrinsics

The stereo calibration should estimate the pose of one camera relative to the other. You should report the relative rotation R and translation T. The baseline is usually the magnitude of the translation vector:

$$B = \|T\|_2.$$

Be careful about units. If your calibration board square size is measured in meters, then the translation vector and baseline should also be in meters.

Coding Suggestions

Your stereo code will likely need:

- a loader that pairs left and right images correctly,
- a detector that only keeps image pairs where the target is found in both views,
- independent calibration functions for the left and right cameras,
- a stereo calibration function that estimates R, T, E, and F,
- rectification maps for the left and right images,
- visualization code for epipolar lines,
- a stereo matching method such as block matching or semi-global block matching,
- a depth conversion function that handles invalid or zero disparity values.

Start by checking that each left image is matched with the correct right image. Incorrect pairing can make stereo calibration fail even if single-camera calibration works well.

Deliverables

- Source code for stereo calibration, rectification, disparity, and depth.
- The left and right camera intrinsic matrices.
- The left and right distortion coefficients.
- The stereo relative rotation R and translation T.
- The stereo baseline length.
- The essential matrix E and fundamental matrix F, or a short discussion of their meaning.
- Rectified stereo image visualizations with epipolar lines.
- Disparity maps.
- Depth maps or reconstructed 3D point visualizations.
- Short written answers:
  - What does the stereo baseline mean?
  - Why do we need stereo rectification?
  - Why does larger disparity correspond to smaller depth?
  - What types of image regions produce unreliable disparity?
  - How does calibration error affect depth estimation?

Target

Your rectified image pairs should have approximately horizontal epipolar correspondences. Your disparity and depth maps do not need to be perfect, but they should show meaningful structure and should be consistent with the equation $Z = f_x B/d$.

Comparison and Analysis

Goal

Analyze the quality of your single-camera and stereo calibration results. The goal is not only to produce calibration parameters, but also to understand what makes calibration good or bad and how calibration quality affects downstream robotic perception tasks.

Suggested Experiments

Choose several experiments that help explain your results. Possible directions include:

- Compare calibration results using different numbers of images.
- Compare results using high-diversity viewpoints versus low-diversity viewpoints.
- Compare calibration with and without distortion parameters.
- Compare checkerboard calibration with AprilTag, ArUco, or ChArUco calibration if you use markers.
- Study how reprojection error changes when blurry or poorly lit images are included.
- Study how stereo baseline affects depth sensitivity.
- Measure the distance to a real object using stereo depth and compare it with a manual measurement.
- Visualize camera poses relative to the calibration board.

You do not need to complete every experiment listed above. Choose experiments that help you make a clear argument about your calibration quality.

Optional Pose Estimation Extension

As an optional extension, use your calibrated single-camera intrinsics to estimate the pose of a calibration board in a new image. This can be done using 2D-3D correspondences and a Perspective-n-Point solver.

For one test image, report the board pose with respect to the camera:

$$^{C}T_{B} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}.$$

You may also visualize the estimated coordinate frame on the image. This optional part is intended to connect calibration with object pose estimation and visual servoing.

Practical Tricks

You are encouraged to try practical calibration tricks and report whether they help. Examples include:

- using more diverse target orientations,
- removing images with failed or inaccurate corner detection,
- refining detected corners to subpixel accuracy,
- using a larger calibration target,
- improving lighting and reducing motion blur,
- using a fixed and carefully measured square size,
- masking invalid disparity values before visualizing depth.

Deliverables

- A comparison of single-camera and stereo calibration quality.
- At least one calibration-quality analysis, such as number of images, viewpoint diversity, or distortion on/off.
- Plots or tables showing reprojection error, baseline, or depth-estimation results.
- A discussion of major error sources.
- A discussion of how calibration affects robotic perception tasks, such as object pose estimation, visual servoing, stereo depth estimation, and hand-eye calibration.
- Short written answers:
  - Which factor affected calibration quality the most?
  - What failure cases did you observe?
  - How would poor calibration affect a robot manipulation system?
  - What would you improve if you collected the calibration dataset again?

Target

The goal is not to obtain the smallest possible reprojection error by over-tuning. The goal is to run a careful calibration pipeline, show meaningful visualizations, and make a clear argument about calibration quality and its effect on robot perception.

Final Report and Submission

Submit a short report with:

1. A short explanation of the pinhole camera model.
2. A short explanation of radial and tangential distortion.
3. The calibrated single-camera intrinsic matrix and distortion coefficients.
4. The average reprojection error and your interpretation of calibration quality.
5. Example images before and after undistortion.
6. The left and right stereo camera intrinsics.
7. The stereo extrinsics R and T, including the baseline length.
8. Stereo rectification visualization with epipolar lines.
9. Disparity and depth visualization.
10. A discussion of major error sources and possible improvements.
11. A short discussion of how calibration quality affects robotic perception tasks.

The report should include the exact commands or scripts used for your main results. If your project uses configuration files, include the relevant configuration names and important parameters such as board size, square size, image resolution, and stereo matching settings.

Submission Package

Please compress all required files into a single `.zip` archive. Use the following naming convention:

`{your student id}-{Name}-perception-calibration.zip`

Your submission should include:

- **Source Code:** Your completed Python implementation in the starter codebase.
- **Patch File:** A Git patch showing all changes relative to the original starter codebase. You can generate it with `git diff > mypatch.patch`.

- **Calibration Data:** The calibration images you used. If a dataset is provided by the instructor, include your processed results and scripts. If you collect your own images, include both the raw images and a short description of your camera setup.
- **Report:** A concise PDF summary of your implementation, calibration results, visualizations, error analysis, and discussion.
- **Demonstration:** Short video, GIF, or image sequence showing detected corners, undistorted images, stereo rectification, disparity, and depth.

Suggested Grading

- Problem 1: single-camera calibration, intrinsics, distortion, extrinsics, and reprojection error, 35 points.
- Problem 2: stereo calibration, stereo extrinsics, rectification, disparity, and depth, 35 points.
- Comparison, calibration-quality analysis, and robotic perception discussion, 15 points.
- Report quality, code quality, and reproducibility, 15 points.

Notes

- Do not hard-code absolute paths that only work on your machine.
- Save all configuration files and important calibration outputs.
- Keep all plots and images readable, with clear labels and captions.
- Use consistent physical units for square size, translation, baseline, and depth.
- Check that left and right stereo images are correctly paired.
- Use visualizations to verify that high numerical accuracy matches reasonable image behavior.
- Your project organization is up to you, but the grader must be able to find and run the submitted code from your instructions.